

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 669 — Computational Optimization

Convexity, duality, and relaxation — the theory underneath everything the last two courses did empirically

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 6 (with CSCE 633 and CSCE 636)

Prerequisites: CSCE 629 Analysis of Algorithms; linear algebra and multivariable calculus; CSCE 633 is helpful for motivation

Effort: 12–14 hours per week · 13 modules + 2 project weeks

Deliverable: A real problem modelled as an optimisation program, solved three ways, with the formulation justified, the dual interpreted, and an honest account of where the model stops matching the problem

Course description

CSCE 633 and CSCE 636 ran gradient descent on things and reported what happened. **This course is about why it works, when it provably works, and what to do when it provably does not.**

The organising distinction is convexity. A convex problem has one minimum, local information suffices to find it, and efficient algorithms with proven convergence rates exist. A non-convex problem has none of those guarantees. **Module 02 is about recognising which you have**, because that single determination tells you more about what is possible than any other property of a problem.

The second theme is **duality**, which is the most useful idea in the subject and the least intuitive. **Every minimisation has a shadow maximisation whose value bounds it**, whose variables price the constraints, and whose solution certifies optimality. **Module 04 develops it for linear programs and Module 06 generalises it**, and the pattern recurs in support vector machines, in network flow, in economics, and in the approximation guarantees of Module 10.

The third is that **modelling is the hard part and the under-taught part**. Solvers are excellent and freely available; the difficulty is turning a messy real objective into a program whose solution means something. **Module 12 is entirely about that translation**, and it is where most of the value in applied optimisation actually sits.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 Recognise convexity and explain what it guarantees.
- 2 Formulate and solve linear programs.
- 3 Construct and interpret a dual, including shadow prices.
- 4 State convergence rates for the first-order methods and what they depend on.
- 5 Apply KKT conditions to constrained problems.
- 6 Explain why interior point methods made linear programming polynomial in practice.
- 7 Explain stochastic gradient convergence and variance reduction.
- 8 Apply branch and bound and LP relaxation to integer programs.
- 9 Derive an approximation guarantee from a relaxation.
- 10 Model a real problem and say where the model fails.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	The Shape of the Problem	What makes one optimisation problem easy and another intractable?
2	Convexity	How do you recognise a convex problem?
3	Linear Programming	What can you do when everything is linear?
4	Duality	What is the dual, and what is it telling you?
5	First-Order Methods	How fast does gradient descent converge, and on what does that depend?
6	Constrained Optimisation	What characterises an optimum subject to constraints?
7	Newton and Interior Point	What does second-order information buy?
8	Stochastic and Large-Scale Methods	What happens when each gradient is only an estimate?
9	Integer Programming	How do solvers handle a problem that is intractable?
10	Approximation and Relaxation	If you cannot solve it exactly, how close can you guarantee?
11	Combinatorial Optimisation	Why are some discrete problems easy?
12	Modelling	How do you turn a real problem into a program?
13	Solvers and Shipping	What should you use, and what can you claim?

Projects

Project 1 — Model it, solve it, dualise it (after Module 7)

Take a real problem, formulate it as a mathematical program, solve it three ways, and interpret the dual.

Requirements

- **A written formulation:** decision variables, objective, constraints, each justified against the real problem.
- **A proof or argument that the problem is or is not convex.**
- Solved by your own implementation, by a general-purpose solver, and by a specialised one if applicable.
- **The dual written out and interpreted in the problem's own terms** — what does each dual variable price?
- **A sensitivity analysis:** which constraints bind, and what would relaxing each be worth?
- Convergence curves on log axes for the iterative method.

Done means

- **The three solutions agree**, or you can explain precisely why they do not.
- **The dual variables interpreted in the problem's vocabulary**, not in mathematical terms — 'an extra hour of machine time is worth \$34'.
- **A statement of which constraints are binding** and what that tells the problem owner.
- **An account of where the model diverges from the real problem**, which is graded and is the point of the exercise.

Project 2 — When it is not convex (after Module 12)

Take a problem that is integer, combinatorial, or genuinely non-convex, and get a usable answer with a bound.

Requirements

- **An integer or combinatorial formulation** of a problem you care about.
- **The LP relaxation solved, and the integrality gap measured.**
- Branch and bound implemented, with the node count reported.
- **At least one cut or one valid inequality** added, with its effect on the node count measured.
- **An approximation algorithm with a proven ratio**, or a heuristic with a measured gap against the bound.
- A comparison against a commercial or open solver.

Done means

- **A bound as well as a solution** — 'within 3% of optimal' is a far stronger claim than a number.
- **Node counts with and without your cuts**, measured.

- **The integrality gap reported**, with a comment on whether the relaxation was tight.
- **An honest comparison against the solver**, which will probably win. Report by how much.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Boyd & Vandenberghe — Convex Optimization (free book) and Stanford EE364A (free lectures)	The primary source. Free book, free video lectures, free problem sets with solutions. Modules 02 through 07 follow its development closely, and the modelling chapter informs Module 12.
MIT 15.093 — Optimization Methods (free, OCW)	Stronger on linear and integer programming than Boyd, which is convex-first. The reference for Modules 03, 09, and 11.
Nocedal & Wright — Numerical Optimization	The reference for the algorithmic detail of Modules 05 and 07 — line searches, trust regions, quasi-Newton. Library copy.
Williamson & Shmoys — The Design of Approximation Algorithms (free PDF)	Free in full, and the reference for Module 10. The LP rounding and randomised rounding chapters are the relevant ones.
CVXPY documentation and examples (free)	Disciplined convex programming — a modelling language that refuses to accept a problem it cannot verify is convex, which is itself instructive.
Bixby — A Brief History of Linear Programming Computation (free)	The Module 13 material: solvers improved by roughly as much from algorithms as from hardware, over the same period.

Tooling

- **Python with CVXPY and SciPy.** CVXPY will refuse a non-convex formulation, which teaches you convexity faster than any exercise.
- **A real solver:** HiGHS (free, excellent) for linear and integer programs; SCS or ECOS for conic problems. Gurobi and Mosek have free academic licences and are substantially faster on hard instances.
- **Implement the core algorithms yourself first** — gradient descent, simplex, branch and bound — then use the solver. The same pattern as the rest of this program.
- **A plotting setup for convergence curves**, on log axes. **Convergence rates are visible as slopes and invisible as tables.**
- **A problem you actually care about.** Module 12's project requires one, and a textbook instance will not expose the modelling difficulties.
- **Exact rational arithmetic available** for checking small instances — **linear programming is numerically delicate** and CSCE 620's lesson applies.

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.